UTS

PEMBELAJARAN MESIN



NAMA : Aimee Jehova Jireh

NIM : 202431076

KELAS : F

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC : 05

ASISTEN : 1. Jonathan Juan Setyanto

2. Muhammad Caisar Gemilang Pratama

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI 2
BAB I 3
PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 4
BAB II 5
LANDASAN TEORI 5
2.1 Pembelajaran Mesin (Machine Learning) 5
2.2 Supervised Learning 5
2.3 Dataset dan Karakteristik Data Nasabah Bank 6
2.4 Preprocessing Data 7
2.5 Linear Regression 9
2.6 Decision Tree 11
2.7 Ketidakseimbangan Kelas (Class Imbalance) 12
2.8 Evaluasi Kinerja Model 13
2.9 Confusion Matrix 18
2.10 Penelitian Terdahulu 19
BAB III 21
HASIL 21
BAB IV 45
PENUTUP 45
DAFTAR PUSTAKA 47

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana karakteristik dataset nasabah bank yang digunakan, meliputi jumlah data, tipe atribut, distribusi variabel, serta kondisi kualitas datanya?
2. Bagaimana hasil Exploratory Data Analysis (EDA) dalam menggambarkan distribusi keputusan nasabah yang menerima (yes) dan tidak menerima (no) penawaran deposito berjangka?
3. Bagaimana tahapan preprocessing yang tepat untuk mempersiapkan fitur numerik dan kategorikal agar dapat digunakan oleh algoritma Linear Regression dan Decision Tree?
4. Bagaimana penerapan Linear Regression terhadap target biner setelah label keputusan nasabah dikodekan menjadi 0 dan 1, serta bagaimana keterbatasan hasil prediksinya?
5. Bagaimana penerapan algoritma Decision Tree untuk mengklasifikasikan keputusan nasabah berdasarkan atribut yang tersedia?
6. Bagaimana perbandingan hasil kedua algoritma berdasarkan metrik evaluasi yang sesuai, terutama ketika distribusi kelas target tidak seimbang?

1.2 Tujuan Masalah

1. Memahami struktur dataset bank customers yang diberikan, termasuk jumlah baris, jumlah kolom, tipe data, dan arti umum atribut.
2. Melaksanakan EDA untuk mengetahui pola data, distribusi target, statistik fitur numerik, serta frekuensi fitur kategorikal.
3. Memeriksa kualitas dataset melalui pemeriksaan nilai kosong dan data duplikat sebelum proses pemodelan.
4. Melakukan preprocessing dengan memisahkan target dan fitur, melakukan encoding terhadap kategori, melakukan scaling pada fitur numerik, serta membagi data secara terstratifikasi.
5. Membangun model Linear Regression sebagai Linear Probability Model dan model Decision Tree untuk memprediksi keputusan nasabah.
6. Mengevaluasi model menggunakan MAE, MSE, RMSE, R^2, accuracy, precision, recall, F1-score, dan confusion matrix sesuai karakteristik masing-masing model.
7. Menganalisis kelebihan dan keterbatasan kedua model berdasarkan hasil pengujian pada data uji

1.3 Manfaat Masalah

1. Menambah pemahaman mengenai alur kerja pembelajaran mesin, mulai dari memahami data, melakukan EDA, preprocessing, pelatihan model, sampai evaluasi.
2. Melatih kemampuan membaca dataset nyata yang terdiri atas atribut numerik dan kategorikal serta memahami perbedaan perlakuan terhadap kedua jenis atribut.
3. Memahami pentingnya menjaga pemisahan data latih dan data uji agar hasil evaluasi tidak bias akibat kebocoran informasi.
4. Mengetahui bahwa accuracy saja belum cukup untuk menilai model pada dataset dengan distribusi kelas yang tidak seimbang.
5. Memahami perbedaan pendekatan regresi linear pada target biner dengan algoritma klasifikasi berbasis pohon keputusan.
6. Memberikan pengalaman dalam menyusun analisis hasil pemodelan secara sistematis dan menghubungkan hasil komputasi dengan teori ilmiah.

BAB II
LANDASAN TEORI

2.1 Pembelajaran Mesin (Machine Learning)

Pembelajaran mesin atau Machine Learning merupakan salah satu cabang dari kecerdasan buatan (Artificial Intelligence) yang memungkinkan komputer mempelajari pola dari sekumpulan data untuk menghasilkan prediksi atau keputusan tanpa harus diberikan aturan secara eksplisit untuk setiap kondisi. Dalam pembelajaran mesin, komputer memanfaatkan data sebagai dasar untuk mengenali hubungan antarvariabel dan membangun suatu model yang dapat digunakan untuk menganalisis data baru.

Secara umum, proses pembelajaran mesin dimulai dari pengumpulan data, dilanjutkan dengan persiapan dan pengolahan data, pembentukan model, pelatihan model, serta evaluasi terhadap hasil prediksi. Kualitas data memiliki pengaruh penting karena model mempelajari pola berdasarkan informasi yang tersedia. Data yang tidak sesuai, tidak konsisten, atau memiliki distribusi yang tidak seimbang dapat memengaruhi kemampuan model dalam menghasilkan prediksi.

Salah satu pendekatan dalam pembelajaran mesin adalah Supervised Learning. Pendekatan ini menggunakan data yang telah memiliki label atau target sebagai bahan pembelajaran. Model mempelajari hubungan antara variabel masukan (features) dengan variabel keluaran (target), kemudian menggunakan pola tersebut untuk memprediksi target dari data yang belum pernah dipelajari.

Dalam praktikum ini, pembelajaran mesin diterapkan untuk memprediksi keputusan nasabah bank dalam menerima atau menolak penawaran deposito berjangka. Data nasabah digunakan sebagai variabel masukan, sedangkan keputusan nasabah yang dinyatakan melalui kategori yes dan no digunakan sebagai target prediksi.

2.2 Supervised Learning

Supervised Learning merupakan metode pembelajaran mesin yang proses pelatihannya menggunakan data berlabel. Setiap data memiliki pasangan antara variabel masukan dan hasil yang ingin diprediksi. Dengan mempelajari pasangan tersebut, model berusaha menemukan pola atau hubungan yang dapat digunakan untuk memperkirakan hasil pada data baru.

Pada Supervised Learning, permasalahan umumnya dibedakan menjadi dua jenis, yaitu regresi dan klasifikasi.

1. Regresi

Regresi digunakan ketika keluaran yang diprediksi berbentuk nilai numerik atau kontinu. Contohnya adalah prediksi harga rumah, jumlah penjualan, dan perkiraan pendapatan.

2. Klasifikasi

Klasifikasi digunakan ketika keluaran yang diprediksi berbentuk kategori atau kelas tertentu. Contohnya adalah menentukan apakah seorang nasabah menerima atau menolak penawaran produk bank.

Pada praktikum ini, target yang digunakan merupakan data kategorikal biner, yaitu yes dan no. Kedua kategori tersebut kemudian direpresentasikan menjadi nilai numerik 1 dan 0 agar dapat diolah oleh algoritma yang digunakan. Model yang dibangun bertujuan mempelajari hubungan antara karakteristik nasabah dengan keputusan terhadap penawaran deposito.

2.3 Dataset dan Karakteristik Data Nasabah Bank

Dataset merupakan kumpulan data yang disusun secara terstruktur dan terdiri atas sejumlah baris serta kolom. Setiap baris umumnya merepresentasikan satu objek atau pengamatan, sedangkan setiap kolom merepresentasikan atribut atau variabel yang menjelaskan objek tersebut.

Dalam pembelajaran mesin, dataset berfungsi sebagai sumber informasi untuk membangun dan mengevaluasi model. Dataset yang digunakan dalam praktikum ini adalah data nasabah bank yang berisi informasi mengenai karakteristik pribadi, kondisi pekerjaan, informasi keuangan, serta riwayat interaksi nasabah dengan pihak bank.

Dataset tersebut memiliki 45.211 baris data dan 17 kolom. Salah satu kolomnya adalah y, yaitu variabel target yang menunjukkan keputusan nasabah terhadap penawaran deposito berjangka. Nilai yes menunjukkan nasabah menerima penawaran, sedangkan no menunjukkan nasabah tidak menerima penawaran.

Karakteristik data dalam dataset dapat dibedakan menjadi dua jenis, yaitu:

1. Data numerik

Data numerik merupakan data yang dinyatakan dalam bentuk angka dan dapat digunakan untuk operasi perhitungan. Contohnya meliputi umur nasabah, saldo rekening, jumlah hari sejak terakhir dihubungi, serta jumlah kontak yang dilakukan.

2. Data kategorikal

Data kategorikal merupakan data yang menunjukkan kelompok atau jenis tertentu. Contohnya adalah pekerjaan, status perkawinan, tingkat pendidikan, jenis kontak, dan bulan terakhir nasabah dihubungi.

Perbedaan karakteristik tersebut perlu diperhatikan karena data numerik dan kategorikal memerlukan proses pengolahan yang berbeda sebelum digunakan dalam pembentukan model pembelajaran mesin.

2.4 Preprocessing Data

Preprocessing data merupakan tahapan persiapan data sebelum digunakan untuk melatih model pembelajaran mesin. Tahapan ini bertujuan memastikan data berada dalam kondisi yang sesuai sehingga dapat diproses oleh algoritma secara lebih efektif. Proses persiapan data dapat mencakup pemeriksaan data, penanganan nilai kosong, transformasi variabel, dan pembagian dataset.

Beberapa tahapan preprocessing yang relevan dengan praktikum ini adalah sebagai berikut.

2.4.1 Pemeriksaan Data

Pemeriksaan data dilakukan untuk mengetahui kondisi awal dataset, seperti jumlah baris, jumlah kolom, tipe data, nilai kosong, dan data duplikat. Tahapan ini penting untuk memahami struktur data serta mengidentifikasi kemungkinan permasalahan yang dapat mengganggu proses pembelajaran model.

Pada dataset nasabah bank, pemeriksaan dilakukan untuk mengetahui karakteristik setiap atribut serta memastikan bahwa data dapat digunakan dalam proses analisis.

2.4.2 Penanganan Nilai Kosong (Missing Value)

Missing value merupakan kondisi ketika suatu atribut tidak memiliki nilai pada pengamatan tertentu. Nilai kosong perlu diperhatikan karena beberapa algoritma pembelajaran mesin tidak dapat langsung memproses data yang memiliki nilai kosong.

Salah satu cara untuk menangani nilai kosong adalah menggunakan imputasi. Pada data numerik, nilai kosong dapat digantikan menggunakan median, sedangkan pada data kategorikal dapat digantikan menggunakan modus atau kategori yang paling sering muncul.

Median digunakan karena relatif tidak mudah dipengaruhi oleh nilai ekstrem. Sementara itu, modus dapat digunakan untuk mempertahankan kategori yang paling umum ditemukan dalam suatu atribut.

2.4.3 Transformasi Data Kategorikal (Encoding)

Sebagian besar algoritma pembelajaran mesin membutuhkan masukan dalam bentuk numerik. Oleh karena itu, data kategorikal perlu diubah menjadi representasi angka melalui proses encoding.

Salah satu teknik yang digunakan adalah One-Hot Encoding. Teknik ini mengubah setiap kategori dalam suatu atribut menjadi kolom indikator tersendiri. Nilai 1 menunjukkan bahwa suatu data termasuk dalam kategori tersebut, sedangkan nilai 0 menunjukkan bahwa data tidak termasuk dalam kategori tersebut.

Teknik ini sesuai untuk atribut kategorikal yang tidak memiliki urutan tingkatan tertentu. Dengan demikian, model tidak menganggap satu kategori memiliki nilai yang lebih tinggi daripada kategori lainnya hanya karena perbedaan angka hasil pengkodean.

2.4.4 Standardisasi Data

Standardisasi merupakan proses mengubah skala data numerik agar memiliki rata-rata mendekati 0 dan standar deviasi mendekati 1. Proses ini dapat membantu algoritma tertentu dalam mengolah variabel yang memiliki rentang nilai berbeda.

Salah satu metode standardisasi adalah StandardScaler, dengan rumus:

$$z = \frac{x - \mu}{\sigma}$$

Keterangan:

- z = nilai setelah standardisasi.
- x= nilai asli.
- μ = nilai rata-rata.
- σ = standar deviasi.

Standardisasi terutama bermanfaat pada algoritma yang sensitif terhadap perbedaan skala variabel, seperti model regresi yang menggunakan proses optimasi tertentu.

2.4.5 Pembagian Data (Data Splitting)

Pembagian data merupakan proses memisahkan dataset menjadi data pelatihan (training data) dan data pengujian (testing data).

Data pelatihan digunakan untuk membangun model, sedangkan data pengujian digunakan untuk mengukur kemampuan model dalam memprediksi data yang tidak digunakan selama pelatihan.

Dalam praktikum ini, pembagian data dilakukan dengan proporsi 80% untuk data pelatihan dan 20% untuk data pengujian. Selain itu, digunakan teknik stratified splitting agar proporsi kelas target pada kedua bagian data tetap mendekati distribusi awalnya.

Teknik tersebut penting karena jumlah nasabah yang menerima penawaran (`yes`) jauh lebih sedikit dibandingkan nasabah yang tidak menerima penawaran (`no`).

2.4.6 Pemilihan Variabel (Feature Selection)

Feature selection merupakan proses menentukan atribut yang digunakan sebagai variabel masukan dalam pembentukan model. Pemilihan atribut dilakukan agar model menggunakan informasi yang relevan dengan tujuan prediksi.

Pada praktikum ini, kolom target `y` tidak dimasukkan sebagai variabel masukan karena kolom tersebut merupakan hasil yang ingin diprediksi. Selain itu, atribut `duration` tidak digunakan karena menunjukkan durasi percakapan antara pihak bank dan nasabah. Informasi tersebut baru diketahui setelah interaksi telepon berlangsung.

Apabila tujuan prediksi adalah memperkirakan keputusan nasabah sebelum proses panggilan dilakukan, penggunaan atribut `duration` dapat menyebabkan data leakage, yaitu kondisi ketika model memperoleh informasi yang seharusnya belum tersedia pada saat prediksi dilakukan. Oleh karena itu, atribut tersebut dikeluarkan agar proses prediksi lebih sesuai dengan kondisi penggunaan sebenarnya.

2.5 Linear Regression

Linear Regression atau regresi linear merupakan salah satu metode statistik dan pembelajaran mesin yang digunakan untuk memodelkan hubungan antara variabel independen dengan variabel dependen. Metode ini berusaha menemukan hubungan linear antara variabel masukan dengan nilai keluaran yang ingin diprediksi.

Secara umum, persamaan regresi linear sederhana dapat dituliskan sebagai berikut:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Keterangan:

- Y= variabel dependen atau target.

- X= variabel independen.
- β_0= konstanta atau intersep.
- β_1 = koefisien regresi.
- ϵ= komponen kesalahan (error).

Untuk regresi linear berganda, persamaannya dapat dituliskan sebagai:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \epsilon$$

Keterangan:

- Y = variabel dependen.
- β_0 = konstanta.
- $\beta_1, \beta_2, \ldots, \beta_n$ = koefisien regresi.
- $X_1, X_2, \ldots, X_n$ = variabel independen.
- ϵ = komponen kesalahan.

Regresi linear berganda melibatkan lebih dari satu variabel independen untuk memperkirakan nilai variabel dependen.

Dalam praktikum ini, Linear Regression digunakan untuk memodelkan target biner yang telah diubah menjadi nilai 0 dan 1. Pendekatan tersebut dikenal sebagai Linear Probability Model (LPM). Nilai prediksi model dapat diinterpretasikan sebagai perkiraan probabilitas suatu kelas, meskipun hasilnya tidak selalu berada dalam rentang 0 sampai 1.

Model regresi linear umumnya menggunakan pendekatan Ordinary Least Squares (OLS), yaitu menentukan koefisien yang meminimalkan jumlah kuadrat selisih antara nilai aktual dan nilai prediksi.

Fungsi objektifnya dapat dituliskan sebagai:

$$\min_{\beta} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Keterangan:

- y_i= nilai aktual.
- $\hat{y}_i$ = nilai prediksi.

- n = jumlah data.

Penggunaan Linear Regression pada target biner perlu memperhatikan keterbatasannya. Model ini tidak secara khusus membatasi nilai prediksi pada rentang probabilitas 0 hingga 1. Oleh sebab itu, hasilnya perlu dievaluasi secara hati-hati sebelum digunakan sebagai dasar pengambilan keputusan.

Lee, Lee, dan Choi (2023) membahas kembali penggunaan Linear Probability Model serta spesifikasi model dalam analisis variabel dependen biner. Kajian tersebut menunjukkan bahwa model probabilitas linear masih dapat digunakan dalam kondisi tertentu, tetapi penerapannya perlu mempertimbangkan karakteristik dan asumsi model.

2.6 Decision Tree

Decision Tree atau pohon keputusan merupakan algoritma pembelajaran mesin yang digunakan untuk melakukan klasifikasi maupun regresi. Algoritma ini bekerja dengan membentuk struktur menyerupai pohon yang terdiri atas simpul keputusan (decision node), cabang (branch), dan simpul akhir (leaf node).

Proses pembentukan pohon dilakukan dengan membagi data berdasarkan atribut tertentu sehingga data pada setiap kelompok menjadi lebih homogen terhadap target yang diprediksi.

Pada permasalahan klasifikasi, algoritma Decision Tree dapat menggunakan beberapa ukuran untuk menentukan pemisahan data, salah satunya adalah Gini Impurity. Ukuran ini digunakan untuk mengetahui tingkat ketidakmurnian suatu kelompok data.

Rumus Gini Impurity adalah:

$$Gini = 1 - \sum_{i=1}^{k} p_i^2$$

Keterangan:

- k= jumlah kelas.
- p_i= proporsi data pada kelas ke-i.

Nilai Gini Impurity yang semakin kecil menunjukkan bahwa kelompok data semakin didominasi oleh satu kelas. Algoritma akan mencari pemisahan yang dapat mengurangi ketidakmurnian kelompok data.

Beberapa konsep penting dalam Decision Tree meliputi:

1. Root Node

Merupakan simpul awal yang menjadi titik pertama pembagian data.

2. Decision Node

Merupakan simpul yang berisi kondisi atau aturan untuk menentukan pembagian data ke cabang berikutnya.

3. Branch

Merupakan cabang yang menghubungkan simpul dan menunjukkan hasil dari suatu kondisi.

4. Leaf Node

Merupakan simpul akhir yang memberikan hasil prediksi kelas atau nilai tertentu.

5. Depth

Merupakan kedalaman pohon yang menunjukkan jumlah tingkatan dari simpul awal hingga simpul tertentu.

Kelebihan Decision Tree adalah hasil model relatif mudah dipahami karena proses pengambilan keputusan dapat direpresentasikan dalam bentuk aturan. Namun, pohon keputusan yang terlalu kompleks berisiko mengalami overfitting, yaitu kondisi ketika model terlalu menyesuaikan diri dengan data pelatihan sehingga kemampuan generalisasinya terhadap data baru menurun.

Untuk mengurangi risiko tersebut, kompleksitas pohon dapat dibatasi melalui pengaturan kedalaman maksimum, jumlah minimum data untuk melakukan pemisahan, serta jumlah minimum data pada setiap simpul akhir.

2.7 Ketidakseimbangan Kelas (Class Imbalance)

Class imbalance merupakan kondisi ketika jumlah data pada setiap kelas dalam suatu dataset memiliki perbedaan yang cukup besar. Kondisi ini dapat memengaruhi proses pelatihan model karena algoritma cenderung lebih mudah mempelajari kelas yang memiliki jumlah data lebih banyak.

Pada dataset nasabah bank yang digunakan dalam praktikum, kelas no memiliki jumlah data yang jauh lebih besar dibandingkan kelas yes. Kondisi tersebut menunjukkan bahwa dataset memiliki distribusi target yang tidak seimbang.

Ketidakseimbangan kelas dapat menyebabkan model menghasilkan akurasi yang terlihat tinggi, tetapi kurang mampu mengenali kelas minoritas. Sebagai contoh, model yang lebih sering memprediksi kelas no dapat memperoleh akurasi tinggi karena sebagian besar data memang termasuk dalam kelas tersebut. Namun, model tersebut belum tentu mampu mengidentifikasi nasabah yang sebenarnya menerima penawaran.

Salah satu pendekatan untuk menangani permasalahan ini adalah penggunaan class weight. Teknik tersebut memberikan bobot yang berbeda terhadap masing-masing kelas sehingga kesalahan prediksi pada kelas minoritas dapat memperoleh perhatian lebih besar selama proses pelatihan.

Pada model Decision Tree dalam praktikum ini digunakan pendekatan class weight balanced. Pendekatan tersebut menyesuaikan bobot kelas berdasarkan frekuensi masing-masing kelas dalam data pelatihan.

Penelitian Watono, Utami, dan Ariatmanto (2024) membahas klasifikasi penerimaan deposito berjangka pada dataset yang tidak seimbang. Penelitian tersebut menunjukkan bahwa ketidakseimbangan kelas merupakan aspek penting yang perlu diperhatikan dalam membangun model prediksi penerimaan deposito bank.

2.8 Evaluasi Kinerja Model

Evaluasi model merupakan proses mengukur kemampuan model dalam menghasilkan prediksi yang sesuai dengan nilai aktual. Evaluasi diperlukan untuk mengetahui tingkat kesalahan, ketepatan prediksi, serta kemampuan model dalam mengenali setiap kelas.

Dalam praktikum ini, evaluasi dilakukan menggunakan metrik regresi dan klasifikasi. Metrik regresi digunakan untuk mengukur selisih nilai prediksi dengan nilai aktual, sedangkan metrik klasifikasi digunakan untuk mengevaluasi hasil pengelompokan kelas.

2.8.1 Mean Absolute Error (MAE)

Mean Absolute Error merupakan metrik yang mengukur rata-rata nilai absolut dari selisih antara hasil prediksi dan nilai aktual.

Rumus MAE:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Keterangan:

- n= jumlah data.
- y_i = nilai aktual.
- $\hat{y}_i$ = nilai prediksi.

Nilai MAE yang semakin kecil menunjukkan bahwa rata-rata kesalahan prediksi semakin rendah.

2.8.2 Mean Squared Error (MSE)

Mean Squared Error merupakan metrik yang menghitung rata-rata kuadrat selisih antara nilai aktual dan nilai prediksi.

Rumus MSE:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Keterangan:

- n = jumlah data.
- y_i = nilai aktual.
- $\hat{y}_i$ = nilai prediksi.

Penggunaan kuadrat menyebabkan kesalahan dengan nilai yang lebih besar mendapatkan penalti lebih tinggi. Semakin kecil nilai MSE, semakin rendah tingkat kesalahan model berdasarkan metrik tersebut.

2.8.3 Root Mean Squared Error (RMSE)

Root Mean Squared Error merupakan akar kuadrat dari MSE. Metrik ini digunakan untuk mengetahui besarnya kesalahan prediksi dalam skala yang sama dengan variabel target.

Rumus RMSE:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Keterangan:

- n= jumlah data.
- y_i= nilai aktual.
- $\hat{y}_i$= nilai prediksi.

Nilai RMSE yang semakin kecil menunjukkan bahwa hasil prediksi model semakin dekat dengan nilai aktual.

2.8.4 R-Squared (R^2)

R-Squared merupakan metrik yang menunjukkan seberapa besar variasi nilai target dapat dijelaskan oleh model.

Rumus R^2:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Keterangan:

- R^2= koefisien determinasi.
- y_i = nilai aktual.
- $\hat{y}_i$= nilai prediksi.
- $\bar{y}$= rata-rata nilai aktual.
- n= jumlah data.

Nilai R^2 yang semakin mendekati 1 menunjukkan kemampuan model yang lebih baik dalam menjelaskan variasi target. Nilai R^2 juga dapat bernilai negatif apabila model menghasilkan kesalahan yang lebih besar dibandingkan prediksi menggunakan rata-rata target.

2.8.5 Accuracy

Accuracy merupakan metrik yang menunjukkan proporsi jumlah prediksi benar terhadap seluruh data yang diuji.

Rumus:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Keterangan :

- TP(True Positive) = data positif yang diprediksi positif dengan benar.
- TN (True Negative) = data negatif yang diprediksi negatif dengan benar.
- FP (False Positive) = data negatif yang salah diprediksi positif.
- FN (False Negative) = data positif yang salah diprediksi negatif.

Nilai accuracy yang tinggi menunjukkan bahwa sebagian besar prediksi model sesuai dengan kelas aktual. Namun, pada dataset yang tidak seimbang, accuracy tidak dapat digunakan sebagai satu-satunya ukuran keberhasilan model.

2.8.6 Precision

Precision merupakan metrik yang mengukur ketepatan model ketika memprediksi suatu data sebagai kelas positif.

Rumus:

$$Precision = \frac{TP}{TP + FP}$$

Keterangan :

- TP(True Positive) = data positif yang diprediksi positif dengan benar.
- TN (True Negative) = data negatif yang diprediksi negatif dengan benar.
- FP (False Positive) = data negatif yang salah diprediksi positif.
- FN (False Negative) = data positif yang salah diprediksi negatif.

Precision yang tinggi menunjukkan bahwa sebagian besar prediksi positif yang dihasilkan model memang merupakan data positif yang sebenarnya.

2.8.7 Recall

Recall merupakan metrik yang mengukur kemampuan model dalam menemukan seluruh data yang benar-benar termasuk kelas positif.

Rumus:

$$Recall = \frac{TP}{TP + FN}$$

Keterangan :

- TP(True Positive) = data positif yang diprediksi positif dengan benar.
- TN (True Negative) = data negatif yang diprediksi negatif dengan benar.
- FP (False Positive) = data negatif yang salah diprediksi positif.
- FN (False Negative) = data positif yang salah diprediksi negatif.

Recall yang tinggi menunjukkan bahwa model mampu mengenali lebih banyak data positif dari seluruh data positif yang tersedia.

2.8.8 F1-Score

F1-Score merupakan nilai rata-rata harmonis antara precision dan recall. Metrik ini digunakan untuk melihat keseimbangan antara ketepatan prediksi positif dan kemampuan model dalam menemukan data positif.

Rumus:

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1-Score dapat menjadi ukuran yang berguna ketika distribusi kelas tidak seimbang dan kedua aspek, yaitu precision serta recall, perlu diperhatikan secara bersamaan.

Keterangan :

- TP(True Positive) = data positif yang diprediksi positif dengan benar.

- TN (True Negative) = data negatif yang diprediksi negatif dengan benar.
- FP (False Positive) = data negatif yang salah diprediksi positif.
- FN (False Negative) = data positif yang salah diprediksi negatif.

2.9 Confusion Matrix

Confusion Matrix merupakan tabel yang digunakan untuk membandingkan hasil prediksi model dengan kelas aktual. Tabel ini membantu mengetahui jumlah prediksi yang benar maupun salah pada setiap kelas.

Dalam klasifikasi biner, Confusion Matrix terdiri atas empat komponen utama:

1. True Positive (TP)

Jumlah data yang sebenarnya termasuk kelas positif dan berhasil diprediksi sebagai kelas positif.

2. True Negative (TN)

Jumlah data yang sebenarnya termasuk kelas negatif dan berhasil diprediksi sebagai kelas negatif.

3. False Positive (FP)

Jumlah data yang sebenarnya termasuk kelas negatif, tetapi diprediksi sebagai kelas positif.

4. False Negative (FN)

Jumlah data yang sebenarnya termasuk kelas positif, tetapi diprediksi sebagai kelas negatif.

Dalam praktikum ini, kelas `yes` ditetapkan sebagai kelas positif, sedangkan kelas `no` menjadi kelas negatif. Dengan demikian, Confusion Matrix dapat menunjukkan seberapa baik model mengenali nasabah yang menerima penawaran deposito serta seberapa banyak nasabah yang salah diklasifikasikan.

Informasi dari Confusion Matrix menjadi dasar untuk menghitung metrik accuracy, precision, recall, dan F1-score. Oleh karena itu, penggunaan Confusion Matrix penting untuk memahami kinerja model secara lebih menyeluruh, terutama ketika dataset memiliki ketidakseimbangan kelas.

2.10 Penelitian Terdahulu

Penelitian terdahulu digunakan sebagai landasan untuk mengetahui penerapan metode pembelajaran mesin pada permasalahan yang memiliki keterkaitan dengan penelitian atau praktikum yang dilakukan. Beberapa penelitian yang relevan dengan prediksi keputusan nasabah bank dan penggunaan algoritma klasifikasi antara lain sebagai berikut.

1. Lee, Lee, dan Choi (2023)

Penelitian berjudul Linear Probability Model Revisited: Why It Works and How It Should Be Specified membahas penggunaan Linear Probability Model untuk menganalisis variabel dependen biner. Penelitian ini relevan dengan praktikum karena target keputusan nasabah memiliki dua kategori, yaitu `yes` dan `no`, yang direpresentasikan dalam bentuk numerik. Kajian tersebut menjadi dasar untuk memahami penggunaan regresi linear pada target biner beserta keterbatasan yang perlu diperhatikan.

2. Delle Site dan Parmar (2024)

Penelitian berjudul On the Linear Probability Model as Binary Choice Random Utility Model membahas model probabilitas linear dalam konteks pilihan biner. Penelitian ini memberikan landasan tambahan mengenai penggunaan pendekatan linear untuk memodelkan keputusan yang terdiri atas dua kemungkinan hasil. Relevansinya terletak pada penerapan model untuk memahami hubungan variabel masukan dengan keputusan nasabah.

3. Maulidah (2023)

Penelitian berjudul Prediksi Peningkatan Jumlah Nasabah Deposito Berjangka Menggunakan Algoritma KNN, Decision Tree, Random Forest dan XGBoost membahas penerapan beberapa algoritma pembelajaran mesin untuk memprediksi peningkatan jumlah nasabah deposito berjangka. Penelitian ini memiliki keterkaitan langsung dengan praktikum karena sama-sama membahas pemanfaatan algoritma pembelajaran mesin dalam permasalahan deposito bank. Penggunaan Decision Tree dalam penelitian tersebut juga menjadi salah satu dasar pemilihan algoritma pada praktikum.

4. Olajide dan Andrew (2023)

Penelitian berjudul Bank Term Deposit Service Patronage Forecasting using Machine Learning membahas pemanfaatan pembelajaran mesin untuk memperkirakan penerimaan layanan deposito berjangka pada sektor perbankan. Penelitian ini relevan karena menunjukkan bahwa data nasabah dapat digunakan untuk membangun model prediksi yang membantu pihak bank memahami kemungkinan penerimaan suatu layanan.

5. Watono, Utami, dan Ariatmanto (2024)

Penelitian berjudul Stacking Ensemble Learning and Instance Hardness Threshold for Bank Term Deposit Acceptance Classification on Imbalanced Dataset membahas klasifikasi penerimaan deposito bank dengan memperhatikan ketidakseimbangan distribusi kelas. Penelitian tersebut relevan dengan praktikum karena dataset nasabah yang digunakan juga memiliki jumlah kelas `no` lebih banyak dibandingkan kelas `yes`. Hal ini menunjukkan pentingnya memperhatikan distribusi target dan menggunakan metrik evaluasi yang tidak hanya berfokus pada akurasi.

6. Nasir et al. (2024)

Penelitian berjudul Data-Driven Decision-Making for Bank Target Marketing Using Supervised Learning Classifiers on Imbalanced Big Data membahas penggunaan algoritma Supervised Learning untuk mendukung pengambilan keputusan dalam pemasaran produk perbankan dengan data yang tidak seimbang. Penelitian ini relevan karena memiliki fokus pada pemanfaatan data nasabah untuk memprediksi respons terhadap kegiatan pemasaran bank. Berdasarkan beberapa penelitian tersebut, dapat diketahui bahwa pembelajaran mesin telah digunakan dalam berbagai permasalahan perbankan, khususnya untuk memprediksi penerimaan produk deposito dan mendukung kegiatan pemasaran. Penggunaan Linear Regression dan Decision Tree dalam praktikum ini bertujuan membandingkan dua pendekatan pemodelan dalam memprediksi keputusan nasabah berdasarkan karakteristik data yang tersedia.

BAB III
HASIL

Langkah 1. Import Library

In [2]:
```
# 202431076_AimeeJehovaJireh

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)
```

Penjelasan Codingan:

Pada tahap pertama dilakukan proses import library atau pemanggilan pustaka Python yang dibutuhkan selama praktikum. Library tersebut digunakan untuk membantu proses pembacaan dataset, pengolahan data, visualisasi, pembangunan model, serta evaluasi hasil prediksi.

- `pandas` digunakan untuk membaca dan mengelola dataset dalam bentuk tabel.
- `numpy` digunakan untuk membantu proses perhitungan numerik.
- `matplotlib.pyplot` digunakan untuk membuat visualisasi data dalam bentuk grafik.
- `train_test_split` digunakan untuk membagi dataset menjadi data latih dan data uji.
- `ColumnTransformer` digunakan untuk menerapkan preprocessing berdasarkan jenis fitur.
- `OneHotEncoder` digunakan untuk mengubah data kategorikal menjadi bentuk numerik.
- `StandardScaler` digunakan untuk melakukan standardisasi data numerik.
- `Pipeline` digunakan untuk menyusun tahapan preprocessing dan pemodelan secara berurutan.
- `SimpleImputer` digunakan untuk menangani kemungkinan nilai kosong.
- `LinearRegression` digunakan untuk membangun model regresi linear.
- `DecisionTreeClassifier` digunakan untuk membangun model klasifikasi berbasis pohon keputusan.
- Library `sklearn.metrics` digunakan untuk menghitung berbagai metrik evaluasi, seperti MAE, MSE, RMSE, R^2, Accuracy, Precision, Recall, dan F1-Score.

Pembahasan Output: Pada tahap ini tidak menghasilkan tabel atau grafik karena proses yang dilakukan hanya memanggil library yang diperlukan. Apabila tidak terdapat pesan kesalahan (error), berarti seluruh library berhasil dipanggil dan siap digunakan pada tahapan berikutnya.

Langkah 2. Membaca Dataset Bank Customer

In [24]:
```
# 202431076_AimeeJehovaJireh

df = pd.read_csv("bank_customers.csv")

print("Dataset berhasil dibaca.")

display(df.head())
```

```
Dataset berhasil dibaca.
```

	age;"job";"marital";"education";"default";"balance";"housing";"loan";"contact";"day";"month";"duration";"campaign";"pdays";"previous";"poutcome";"y"
0	58;"management";"married";"tertiary";"no";2143...
1	44;"technician";"single";"secondary";"no";29;"...
2	33;"entrepreneur";"married";"secondary";"no";2...
3	47;"blue-collar";"married";"unknown";"no";1506...
4	33;"unknown";"single";"unknown";"no";1;"no";"n...

Penjelasan Coding:

Pada tahap ini dilakukan proses membaca dataset nasabah bank yang telah diekstrak sebelumnya dari file ZIP.

1. pd.read_csv() digunakan untuk membaca file dengan format CSV.
2. "bank_customers.csv" merupakan nama file dataset yang digunakan dalam praktikum.
3. Hasil pembacaan disimpan ke dalam variabel df dalam bentuk DataFrame.
4. df.head() digunakan untuk menampilkan lima baris pertama dataset sebagai pemeriksaan awal.

Pembahasan Output:

Berdasarkan hasil pembacaan, dataset berhasil dimuat ke dalam DataFrame. Lima baris pertama ditampilkan untuk memastikan bahwa file dapat dibaca dengan benar dan setiap atribut telah dikenali oleh Python.

Dataset tersebut berisi informasi mengenai karakteristik nasabah bank yang nantinya digunakan sebagai dasar dalam membangun model prediksi keputusan penerimaan penawaran deposito berjangka.

Langkah 3. Memeriksa Ukuran dan Struktur Dataset

In [4]:
```
# 202431076_AimeeJehovaJireh

print("Ukuran dataset:", df.shape)

print("\nInformasi dataset:")
df.info()

print("\nStatistik deskriptif:")
display(df.describe(include="all").T)
```

```
Ukuran dataset: (45211, 1)

Informasi dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 1 columns):
 #   Column
Non-Null Count  Dtype
---  ------
--------------  -----
 0   age;"job";"marital";"education";"default";"balance";"housing";"loan";"contact";"day";"month";"duration";"campaign";"pda
ys";"previous";"poutcome";"y"  45211 non-null  object
dtypes: object(1)
memory usage: 353.3+ KB

Statistik deskriptif:
```

	count	u
age;"job";"marital";"education";"default";"balance";"housing";"loan";"contact";"day";"month";"duration";"campaign";"pdays";"previous";"poutcome";"y"	45211	

Penjelasan Coding:

Tahap ini bertujuan untuk mengetahui ukuran, struktur, dan karakteristik awal dataset.

1. df.shape digunakan untuk mengetahui jumlah baris dan kolom pada dataset.
2. df.info() digunakan untuk menampilkan informasi setiap kolom, seperti nama atribut, tipe data, dan jumlah nilai yang tersedia.
3. df.describe(include="all") digunakan untuk menghasilkan statistik deskriptif dari atribut numerik maupun kategorikal.
4. .T digunakan untuk mengubah posisi tabel agar nama setiap atribut ditampilkan dalam baris sehingga lebih mudah dibaca.

Pembahasan Output:

Berdasarkan hasil pemeriksaan, dataset memiliki 45.211 baris dan 17 kolom. Setiap baris merepresentasikan satu catatan data nasabah, sedangkan setiap kolom menunjukkan atribut yang menggambarkan karakteristik nasabah atau informasi kegiatan pemasaran bank.

Dataset terdiri atas atribut numerik dan kategorikal. Informasi ini menjadi dasar untuk menentukan metode preprocessing yang sesuai sebelum model dilatih.

Langkah 4. Memeriksa Missing Value dan Data Duplikat

In [5]:
```
# 202431076_AimeeJehovaJireh

print("Missing value per kolom:")
display(df.isnull().sum())

print("Jumlah baris duplikat:", df.duplicated().sum())
```

```
Missing value per kolom:

age;"job";"marital";"education";"default";"balance";"housing";"loan";"contact";"day";"month";"duration";"campaign";"pday
s";"previous";"poutcome";"y"    0
dtype: int64

Jumlah baris duplikat: 0
```

Penjelasan Coding

Tahap ini dilakukan untuk memeriksa kualitas data, terutama keberadaan nilai kosong dan baris yang tercatat lebih dari satu kali.

1. df.isnull() digunakan untuk mendeteksi nilai kosong pada setiap sel dataset.
2. .sum() digunakan untuk menghitung jumlah nilai kosong pada masing-masing kolom.
3. df.duplicated() digunakan untuk mendeteksi baris yang memiliki isi sama dengan baris sebelumnya.
4. .sum() digunakan kembali untuk menghitung jumlah keseluruhan baris duplikat.

Pembahasan Output

Berdasarkan hasil pemeriksaan, tidak ditemukan missing value pada seluruh kolom dataset. Selain itu, jumlah baris duplikat juga menunjukkan nilai 0.

Hasil tersebut menunjukkan bahwa dataset memiliki kelengkapan data yang baik berdasarkan pemeriksaan nilai kosong dan duplikasi. Dengan demikian, tidak diperlukan penghapusan baris akibat kedua permasalahan tersebut.

Perlu diperhatikan bahwa kategori unknown yang terdapat pada beberapa atribut kategorikal tetap dianggap sebagai nilai kategori yang tercatat, bukan sebagai missing value.

Langkah 5. Menganalisis Distribusi Variabel Target

In [8]:

```
# 202431076_AimeeJehovaJireh

# Membaca dataset yang sudah diekstrak
df = pd.read_csv(
    "bank_customers.csv",
    sep=None,
    engine="python",
    encoding="utf-8-sig"
)

# Merapikan nama kolom
df.columns = (
    df.columns.astype(str)
    .str.strip()
    .str.replace("\ufeff", "", regex=False)
    .str.lower()
)

# Memastikan kolom target tersedia
if "y" not in df.columns:
    raise ValueError(
        f"Kolom target 'y' tidak ditemukan. "
        f"Kolom yang tersedia: {df.columns.tolist()}"
    )

# Menghitung distribusi target
target_counts = (
    df["y"].astype(str).str.strip().str.lower().value_counts()
)
```

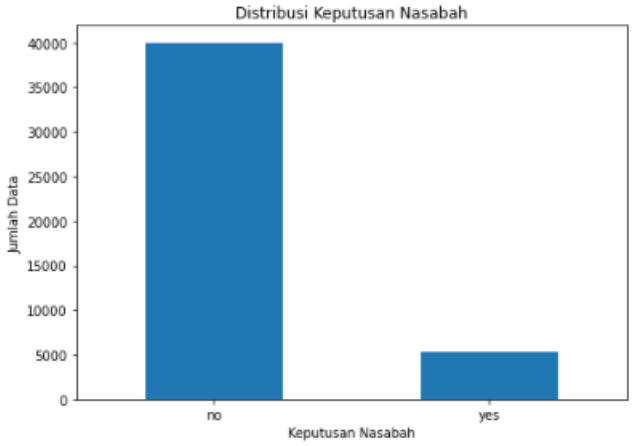
```
print("Distribusi Variabel Target:")
display(target_counts.reindex(["no", "yes"], fill_value=0))

# Membuat grafik distribusi target
target_counts.reindex(["no", "yes"], fill_value=0).plot(
    kind="bar",
    figsize=(7, 5)
)

plt.title("Distribusi Keputusan Nasabah")
plt.xlabel("Keputusan Nasabah")
plt.ylabel("Jumlah Data")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
Distribusi Variabel Target:

y
no     39922
yes     5289
Name: count, dtype: int64
```



Penjelasan: Tahap ini digunakan untuk mengetahui jumlah nasabah yang menerima dan tidak menerima penawaran deposito. value_counts() menghitung jumlah masing-masing kategori, sedangkan plt.bar() membuat grafik batang untuk menggambarkan distribusinya.

Pembahasan Output: Berdasarkan dataset yang digunakan, kategori no berjumlah 39.922 data atau sekitar 88,30%, sedangkan kategori yes berjumlah 5.289 data atau sekitar 11,70%.

Perbedaan jumlah tersebut menunjukkan adanya ketidakseimbangan kelas (class imbalance). Kondisi ini perlu diperhatikan karena model dapat lebih mudah mengenali kelas mayoritas dibandingkan kelas minoritas.

Langkah 6. Exploratory Data Analysis (EDA) Fitur Numerik

In [10]:

```python
# 202431076_AimeeJehovaJireh

# Mengambil seluruh kolom numerik
numeric_cols = df.select_dtypes(
    include=["number"]
).columns.tolist()

# Menampilkan statistik deskriptif
print("Statistik Deskriptif Fitur Numerik:")
display(df[numeric_cols].describe().T)

# Menentukan jumlah grafik
jumlah_fitur = len(numeric_cols)
jumlah_kolom = 3
jumlah_baris = int(np.ceil(jumlah_fitur / jumlah_kolom))

# Membuat area grafik
fig, axes = plt.subplots(
    jumlah_baris,
    jumlah_kolom,
    figsize=(15, 4 * jumlah_baris)
)

# Mengubah axes menjadi array satu dimensi
axes = np.array(axes).reshape(-1)

# Membuat histogram setiap fitur
for i, kolom in enumerate(numeric_cols):

    data = pd.to_numeric(
        df[kolom],
        errors="coerce"
    ).dropna()

    axes[i].hist(
        data,
        bins=25,
        edgecolor="black"
    )

    axes[i].set_title(f"Distribusi {kolom}")
    axes[i].set_xlabel(kolom)
    axes[i].set_ylabel("Frekuensi")

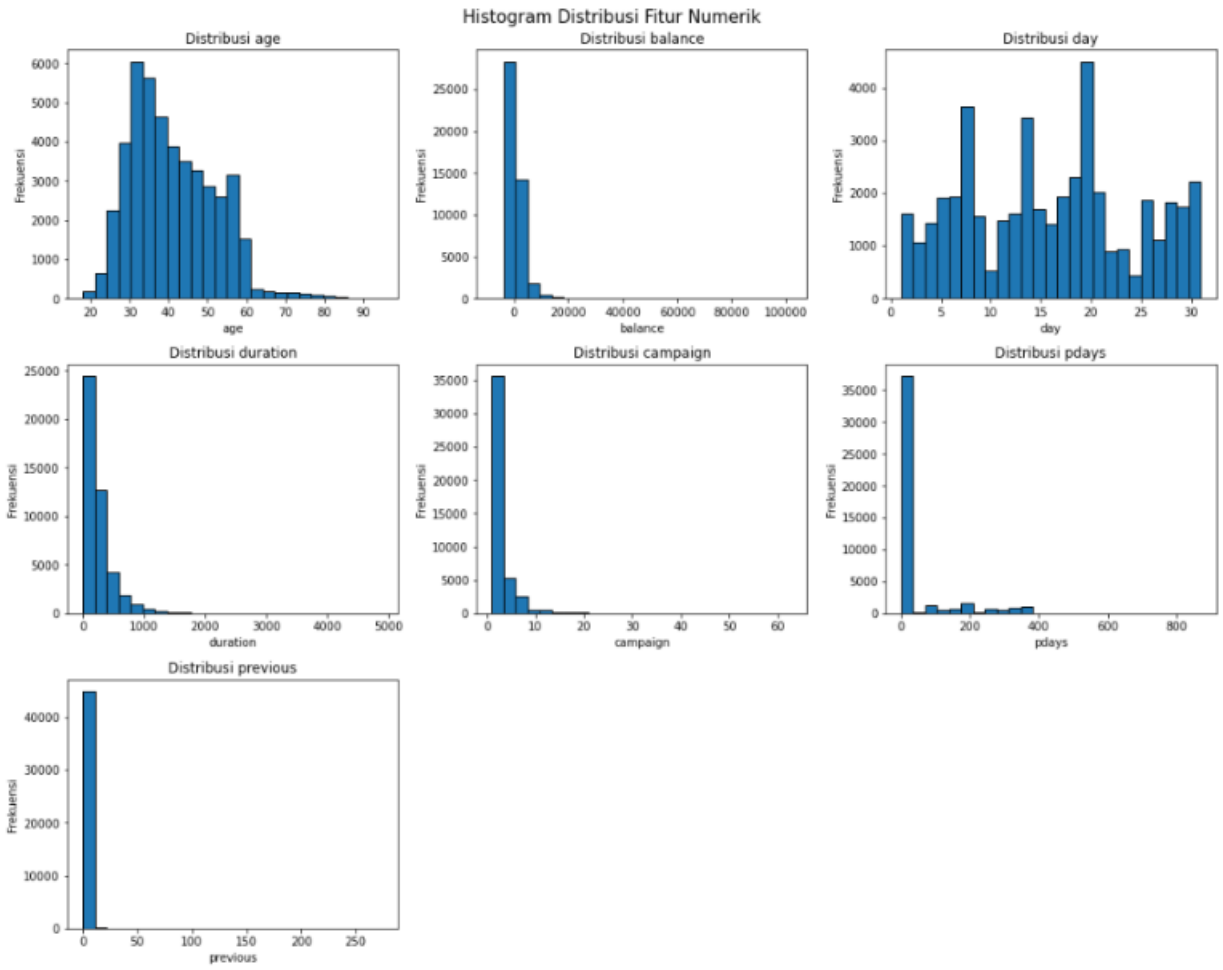
# Menghapus area grafik yang tidak digunakan
for j in range(jumlah_fitur, len(axes)):
    fig.delaxes(axes[j])

plt.suptitle(
    "Histogram Distribusi Fitur Numerik",
    fontsize=15
)

plt.tight_layout()
plt.show()
```

Statistik Deskriptif Fitur Numerik:

	count	mean	std	min	25%	50%	75%	max
age	45211.0	40.936210	10.618762	18.0	33.0	39.0	48.0	95.0
balance	45211.0	1362.272058	3044.765829	-8019.0	72.0	448.0	1428.0	102127.0
day	45211.0	15.806419	8.322476	1.0	8.0	16.0	21.0	31.0
duration	45211.0	258.163080	257.527812	0.0	103.0	180.0	319.0	4918.0
campaign	45211.0	2.763841	3.098021	1.0	1.0	2.0	3.0	63.0
pdays	45211.0	40.197828	100.128746	-1.0	-1.0	-1.0	-1.0	871.0
previous	45211.0	0.580323	2.303441	0.0	0.0	0.0	0.0	275.0



Penjelasan Codingan:

- select_dtypes(include=["number"]) digunakan untuk memilih atribut numerik.
- describe().T digunakan untuk menampilkan nilai rata-rata, standar deviasi, minimum, maksimum, dan kuartil.
- len(numeric_cols) digunakan untuk menghitung jumlah fitur numerik.
- np.ceil() digunakan untuk menentukan jumlah baris grafik agar semua fitur dapat ditampilkan.
- plt.subplots() digunakan untuk menyediakan area visualisasi.
- np.array(axes).reshape(-1) digunakan untuk menyusun seluruh objek grafik menjadi satu dimensi agar mudah diakses.
- Perulangan for digunakan untuk membuat histogram setiap fitur secara otomatis.
- pd.to_numeric() memastikan data yang divisualisasikan berbentuk numerik.
- dropna() mengabaikan nilai kosong apabila ditemukan pada saat visualisasi.
- axes[i].hist() digunakan untuk membuat histogram.
- fig.delaxes() menghapus area grafik yang tidak terpakai.
- plt.tight_layout() digunakan untuk merapikan tampilan grafik.

Pembahasan Output:

Hasil EDA menampilkan statistik deskriptif dan histogram dari masing-masing fitur numerik. Histogram digunakan untuk melihat penyebaran nilai, frekuensi, dan pola distribusi setiap atribut.

Melalui visualisasi tersebut, dapat diketahui bahwa masing-masing atribut memiliki rentang nilai dan karakteristik distribusi yang berbeda. Oleh karena itu, standardisasi diperlukan dalam proses preprocessing, khususnya untuk menyamakan skala fitur numerik.

Perlu diperhatikan bahwa pada tahap EDA ini seluruh fitur numerik ditampilkan, termasuk duration. Namun, duration nantinya tidak digunakan sebagai fitur pemodelan.

Langkah 7. Exploratory Data Analysis (EDA) Fitur Kategorikal

```
In [11]: # 202431076_AimeeJehovaJireh

categorical_cols = df.select_dtypes(
    exclude=["number"]
).columns.tolist()

# Mengeluarkan target dari analisis fitur
categorical_cols = [
    kolom for kolom in categorical_cols
    if kolom != "y"
]

for kolom in categorical_cols:

    print("\nDistribusi Kategori:", kolom)

    display(
        df[kolom]
        .value_counts()
        .head(15)
    )
```

```
Distribusi Kategori: job

job
blue-collar      9732
management       9458
technician       7597
admin.           5171
services         4154
retired          2264
self-employed    1579
entrepreneur     1487
unemployed       1303
housemaid        1240
student           938
unknown           288
Name: count, dtype: int64
```

```
Distribusi Kategori: marital

marital
married     27214
single      12790
divorced     5207
Name: count, dtype: int64
```

```
Distribusi Kategori: education

education
secondary    23202
tertiary     13301
primary       6851
unknown       1857
Name: count, dtype: int64
```

```
Distribusi Kategori: default

default
no     44396
yes      815
Name: count, dtype: int64
```

```
Distribusi Kategori: housing

housing
yes    25130
no     20081
Name: count, dtype: int64
```

```
Distribusi Kategori: loan

loan
no     37967
yes     7244
Name: count, dtype: int64
```

```
Distribusi Kategori: contact

contact
cellular     29285
unknown      13020
telephone     2906
Name: count, dtype: int64
```

```
Distribusi Kategori: month

month
may    13766
jul     6895
aug     6247
jun     5341
nov     3970
apr     2932
feb     2649
jan     1403
oct      738
sep      579
mar      477
dec      214
Name: count, dtype: int64
```

```
Distribusi Kategori: poutcome

poutcome
unknown    36959
failure     4901
other       1840
success     1511
Name: count, dtype: int64
```

Penjelasan: Tahap ini digunakan untuk mengetahui variasi kategori pada fitur non-numerik. select_dtypes() memilih kolom kategorikal, sedangkan value_counts() menghitung frekuensi setiap kategori. Target y tidak dimasukkan karena telah dianalisis pada langkah sebelumnya.

Pembahasan Output: Hasil analisis memperlihatkan variasi kategori pada atribut seperti pekerjaan, status pernikahan, pendidikan, jenis komunikasi, bulan, dan hasil kampanye sebelumnya. Informasi ini digunakan untuk memahami karakteristik data sebelum dilakukan proses One-Hot Encoding.

Langkah 8. Menentukan Fitur dan Target

In [12]:
```
# 202431076_AimeeJehovaJireh

# Mengubah target menjadi angka
y = (
    df["y"]
    .map({"no": 0, "yes": 1})
    .astype(int)
)

# Menghapus target dan duration dari fitur
X = df.drop(
    columns=["y", "duration"]
)

print("Ukuran Fitur:", X.shape)
print("Ukuran Target:", y.shape)

print("\nDistribusi Target:")
display(y.value_counts())
```

```
Ukuran Fitur: (45211, 15)
Ukuran Target: (45211,)

Distribusi Target:

y
0    39922
1     5289
Name: count, dtype: int64
```

Penjelasan: Variabel y digunakan sebagai target yang akan diprediksi. Kategori no diubah menjadi 0, sedangkan yes diubah menjadi 1. Variabel X berisi fitur yang digunakan sebagai masukan model.

Kolom duration dikeluarkan karena informasi lama percakapan baru diketahui setelah panggilan selesai. Penggunaannya dapat menyebabkan data leakage apabila tujuan prediksi dilakukan sebelum panggilan berlangsung.

Pembahasan Output: Hasil pemisahan menghasilkan 15 fitur dan satu variabel target. Target terdiri atas dua kelas, yaitu 0 untuk no dan 1 untuk yes.

Langkah 9. Memisahkan Fitur Numerik dan Kategorikal

```
In [13]: # 202431076_AimeeJehovaJireh

numeric_features = X.select_dtypes(
    include=["number"]
).columns.tolist()

categorical_features = X.select_dtypes(
    exclude=["number"]
).columns.tolist()

print("Fitur Numerik:")
print(numeric_features)

print("\nFitur Kategorikal:")
print(categorical_features)

print("\nJumlah Fitur Numerik:", len(numeric_features))
print("Jumlah Fitur Kategorikal:", len(categorical_features))
```

```
Fitur Numerik:
['age', 'balance', 'day', 'campaign', 'pdays', 'previous']

Fitur Kategorikal:
['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']

Jumlah Fitur Numerik: 6
Jumlah Fitur Kategorikal: 9
```

Langkah 10. Membagi Data Latih dan Data Uji

```
In [14]: # 202431076_AimeeJehovaJireh

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Jumlah Data Latih:", len(X_train))
print("Jumlah Data Uji:", len(X_test))

print("\nDistribusi Data Latih:")
display(y_train.value_counts())

print("\nDistribusi Data Uji:")
display(y_test.value_counts())
```

```
Jumlah Data Latih: 36168
Jumlah Data Uji: 9043

Distribusi Data Latih:

y
0    31937
1     4231
Name: count, dtype: int64


Distribusi Data Uji:

y
0    7985
1    1058
Name: count, dtype: int64
```

Penjelasan codingan:

1. X_train, X_test, y_train, y_test = train_test_split(...)

Digunakan untuk membagi dataset menjadi empat bagian, yaitu:

- X_train: data fitur yang digunakan untuk melatih model.
- X_test: data fitur yang digunakan untuk menguji model.

- y_train: target atau label yang digunakan selama pelatihan.
- y_test: target atau label sebenarnya yang digunakan untuk mengevaluasi hasil prediksi.

2. X

Merupakan kumpulan fitur atau atribut nasabah yang digunakan sebagai masukan model, seperti usia (age), saldo (balance), pekerjaan (job), dan atribut lainnya.

3. y

Merupakan variabel target yang ingin diprediksi, yaitu keputusan nasabah dalam menerima atau menolak penawaran deposito. Nilainya sudah diubah menjadi 0 dan 1.

4. test_size=0.20

Menentukan bahwa 20% dari keseluruhan dataset digunakan sebagai data uji, sedangkan 80% sisanya digunakan sebagai data latih.

5. random_state=42

Digunakan untuk menjaga agar pembagian dataset menghasilkan susunan yang konsisten ketika program dijalankan kembali.

6. stratify=y

Digunakan untuk mempertahankan proporsi kelas target no dan yes pada data latih maupun data uji. Hal ini penting karena dataset memiliki jumlah kelas yang tidak seimbang.

7. len(X_train)

Digunakan untuk menghitung jumlah baris pada data latih.

8. len(X_test)

Digunakan untuk menghitung jumlah baris pada data uji.

9. y_train.value_counts() dan y_test.value_counts()

Digunakan untuk menampilkan jumlah masing-masing kelas target pada data latih dan data uji.

Pembahasan Output

Berdasarkan hasil pembagian, dataset yang berjumlah 45.211 baris dibagi menjadi 36.168 data latih dan 9.043 data uji.

Data latih digunakan agar algoritma dapat mempelajari pola hubungan antara karakteristik nasabah dengan keputusan penerimaan penawaran deposito. Sementara itu, data uji digunakan untuk mengetahui kemampuan model dalam memprediksi data yang belum digunakan selama proses pelatihan.

Penggunaan parameter stratify=y bertujuan agar perbandingan kelas no dan yes tetap terjaga pada kedua bagian dataset.

Langkah 11. Membuat Preprocessing Pipeline

In [15]:
```
# 202431076_AimeeJehovaJireh

# Pipeline fitur numerik
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Pipeline fitur kategorikal
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# Menggabungkan seluruh preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", numeric_transformer, numeric_features),
        ("categorical", categorical_transformer, categorical_features)
    ]
)

print("Preprocessing pipeline berhasil dibuat.")
```

```
Preprocessing pipeline berhasil dibuat.
```

Penjelasan codingan:

1. numeric_transformer = Pipeline(steps=[...])

Digunakan untuk membuat rangkaian preprocessing khusus bagi fitur numerik. Fitur numerik merupakan atribut yang berbentuk angka, seperti age, balance, dan campaign.

2. SimpleImputer(strategy="median")

Digunakan untuk menangani nilai kosong pada fitur numerik dengan menggantinya menggunakan nilai median atau nilai tengah.

Median dipilih karena relatif tidak mudah dipengaruhi oleh nilai ekstrem dibandingkan rata-rata (mean).

3. StandardScaler()

Digunakan untuk melakukan standardisasi data numerik agar fitur memiliki skala yang lebih sebanding.

Contohnya, atribut age memiliki rentang angka yang berbeda dengan balance. Standardisasi membantu menyamakan skala keduanya.

4. categorical_transformer = Pipeline(steps=[...])

Digunakan untuk membuat rangkaian preprocessing khusus bagi fitur kategorikal, seperti job, marital, dan education.

5. SimpleImputer(strategy="most_frequent")

Digunakan untuk mengganti nilai kosong pada fitur kategorikal menggunakan kategori yang paling sering muncul atau modus.

6. OneHotEncoder(handle_unknown="ignore")

Digunakan untuk mengubah kategori berbentuk teks menjadi representasi numerik.

Sebagai contoh, atribut marital yang memiliki kategori single, married, dan divorced akan diubah menjadi beberapa kolom indikator.

Parameter handle_unknown="ignore" digunakan agar kategori baru yang tidak ditemukan ketika pelatihan tidak menyebabkan program mengalami error saat prediksi.

7. ColumnTransformer(transformers=[...])

Digunakan untuk menggabungkan preprocessing numerik dan kategorikal dalam satu objek.

8. ("numeric", numeric_transformer, numeric_features)

Menunjukkan bahwa proses numeric_transformer diterapkan pada seluruh kolom yang terdapat dalam numeric_features.

9. ("categorical", categorical_transformer, categorical_features)

Menunjukkan bahwa proses categorical_transformer diterapkan pada seluruh kolom yang terdapat dalam categorical_features.

10. print()

Digunakan untuk menampilkan pesan bahwa objek preprocessing telah berhasil dibuat.

Pembahasan Output:

Pada tahap ini, program menghasilkan objek preprocessing yang akan digunakan sebelum data dimasukkan ke dalam algoritma pembelajaran mesin.

Preprocessing dilakukan secara berbeda berdasarkan jenis datanya. Fitur numerik diproses menggunakan imputasi median dan standardisasi, sedangkan fitur kategorikal diproses menggunakan imputasi modus dan One-Hot Encoding.

Seluruh proses transformasi nantinya dipelajari dari data latih dan diterapkan pada data uji melalui pipeline model. Dengan demikian, informasi dari data uji tidak digunakan untuk mempelajari parameter preprocessing.

Langkah 12. Melatih Model Linear Regression

In [17]:
```
# 202431076_AimeeJehovaJireh

linear_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

# Melatih model
linear_model.fit(X_train, y_train)

# Prediksi data uji
linear_pred = linear_model.predict(X_test)

# Mengubah prediksi menjadi kelas
linear_class = (
    linear_pred >= 0.5
).astype(int)

print("Lima Prediksi Linear Regression:")
print(linear_pred[:5])

print("\nLima Kelas Prediksi:")
print(linear_class[:5])
```

```
Lima Prediksi Linear Regression:
[-0.03588867  0.01313782  0.03382874 -0.01035309  0.1343689 ]

Lima Kelas Prediksi:
[0 0 0 0 0]
```

Penjelasan codingan:

1. linear_model = Pipeline(steps=[...])

Digunakan untuk membuat model Linear Regression yang memiliki dua tahapan, yaitu preprocessing dan proses regresi.

2. ("preprocessor", preprocessor)

Menunjukkan bahwa data harus melalui proses preprocessing terlebih dahulu sebelum digunakan oleh algoritma.

3. ("regressor", LinearRegression())

Menentukan algoritma yang digunakan, yaitu Linear Regression. Algoritma ini mempelajari hubungan antara fitur nasabah dengan nilai target numerik 0 dan 1.

4. linear_model.fit(X_train, y_train)

Digunakan untuk melatih model menggunakan data latih. Pada proses ini, model mempelajari pola hubungan antara karakteristik nasabah dan keputusan penerimaan deposito.

5. linear_model.predict(X_test)

Digunakan untuk menghasilkan prediksi berdasarkan fitur pada data uji. Hasilnya disimpan ke dalam variabel linear_pred.

6. linear_pred >= 0.5

Digunakan untuk membandingkan setiap nilai prediksi dengan ambang batas 0,5.

- Jika nilai prediksi minimal 0,5, hasilnya dikategorikan sebagai 1 atau yes.
- Jika nilai prediksi kurang dari 0,5, hasilnya dikategorikan sebagai 0 atau no.

7. .astype(int)

Digunakan untuk mengubah hasil perbandingan Boolean (True dan False) menjadi bilangan bulat 1 dan 0.

8. linear_pred[:5]

Digunakan untuk menampilkan lima nilai prediksi pertama yang dihasilkan oleh Linear Regression.

9. linear_class[:5]

Digunakan untuk menampilkan lima hasil prediksi yang sudah diubah menjadi kategori 0 dan 1.

Pembahasan Output:

Pada tahap ini, model Linear Regression menghasilkan nilai prediksi berbentuk angka kontinu. Karena target yang digunakan terdiri atas dua kelas, hasil prediksi tersebut dikonversi menjadi kelas 0 dan 1 menggunakan ambang batas 0,5.

Pendekatan ini dikenal sebagai Linear Probability Model. Namun, perlu diketahui bahwa prediksi Linear Regression tidak selalu berada di antara 0 dan 1. Oleh karena itu, hasil prediksi perlu diperhatikan ketika digunakan untuk klasifikasi.

Langkah 13. Evaluasi Model Linear Regression

In [18]:
```
# 202431076_AimeeJehovaJireh

# Evaluasi regresi
linear_mae = mean_absolute_error(y_test, linear_pred)
linear_mse = mean_squared_error(y_test, linear_pred)
linear_rmse = np.sqrt(linear_mse)
linear_r2 = r2_score(y_test, linear_pred)

# Evaluasi klasifikasi
linear_accuracy = accuracy_score(y_test, linear_class)

linear_precision = precision_score(
    y_test, linear_class, zero_division=0
)

linear_recall = recall_score(
    y_test, linear_class, zero_division=0
)

linear_f1 = f1_score(
    y_test, linear_class, zero_division=0
)

print("EVALUASI LINEAR REGRESSION")
print("--------------------------------")
print("MAE       :", linear_mae)
print("MSE       :", linear_mse)
print("RMSE      :", linear_rmse)
print("R2        :", linear_r2)
print("Accuracy :", linear_accuracy)
print("Precision:", linear_precision)
print("Recall    :", linear_recall)
print("F1-Score :", linear_f1)

print("\nRentang Prediksi:")
print(linear_pred.min(), "sampai", linear_pred.max())
```

```
EVALUASI LINEAR REGRESSION
--------------------------------
MAE      : 0.17468863262297582
MSE      : 0.0859439304723059
RMSE     : 0.2931619526342153
R2       : 0.16808360184208992
Accuracy : 0.8929558774742895
Precision: 0.6470588235294118
Recall   : 0.18714555765595464
F1-Score : 0.2903225806451613

Rentang Prediksi:
-0.2149505615234375 sampai 0.9317550659179688
```

Penjelasan codingan:

Bagian A – Evaluasi regresi

1. mean_absolute_error(y_test, linear_pred)

Digunakan untuk menghitung MAE (Mean Absolute Error), yaitu rata-rata selisih absolut antara nilai aktual dan nilai prediksi.

2. mean_squared_error(y_test, linear_pred)

Digunakan untuk menghitung MSE (Mean Squared Error), yaitu rata-rata kuadrat selisih antara nilai aktual dan nilai prediksi.

3. np.sqrt(linear_mse)

Digunakan untuk menghitung RMSE (Root Mean Squared Error) dengan mencari akar kuadrat dari nilai MSE.

4. r2_score(y_test, linear_pred)

Digunakan untuk menghitung R^2 atau koefisien determinasi yang menunjukkan seberapa baik model menjelaskan variasi nilai target.

Bagian B – Evaluasi klasifikasi

5. accuracy_score(y_test, linear_class)

Digunakan untuk menghitung persentase prediksi yang benar dibandingkan seluruh data uji.

6. precision_score(y_test, linear_class, zero_division=0)

Digunakan untuk menghitung ketepatan model ketika memprediksi kelas positif atau yes.

7. recall_score(y_test, linear_class, zero_division=0)

Digunakan untuk menghitung kemampuan model dalam menemukan nasabah yang sebenarnya termasuk kelas yes.

8. f1_score(y_test, linear_class, zero_division=0)

Digunakan untuk menghitung keseimbangan antara nilai precision dan recall.

9. zero_division=0

Digunakan untuk menentukan nilai 0 apabila perhitungan metrik mengalami pembagian dengan nol, misalnya ketika tidak ada prediksi positif.

Bagian C – Menampilkan hasil

10. print()

Digunakan untuk menampilkan seluruh nilai evaluasi agar hasil pengujian dapat dibaca.

11. linear_pred.min()

Digunakan untuk mencari nilai prediksi paling rendah.

12. linear_pred.max()

Digunakan untuk mencari nilai prediksi paling tinggi.

Pembahasan Output:

Evaluasi dilakukan menggunakan dua kelompok metrik. Kelompok pertama, yaitu MAE, MSE, RMSE, dan R^2, digunakan untuk menilai hasil regresi. Kelompok kedua, yaitu Accuracy, Precision, Recall, dan F1-Score, digunakan untuk menilai hasil klasifikasi setelah nilai prediksi dikonversi menjadi kelas 0 dan 1.

Nilai accuracy yang tinggi tidak selalu menunjukkan bahwa model mampu mengenali seluruh kelas dengan baik. Oleh karena itu, nilai recall dan F1-score juga perlu diperhatikan, terutama karena jumlah nasabah yang menerima penawaran deposito lebih sedikit daripada yang menolak.

Langkah 14. Melatih Model Decision Tree

In [19]:
```
# 202431076_AimeeJehovaJireh

tree_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", DecisionTreeClassifier(
        max_depth=8,
        min_samples_split=20,
        min_samples_leaf=10,
        class_weight="balanced",
        random_state=42
    ))
])

# Melatih model
tree_model.fit(X_train, y_train)

# Melakukan prediksi
tree_pred = tree_model.predict(X_test)

print("Lima Prediksi Decision Tree:")
print(tree_pred[:5])
```

```
Lima Prediksi Decision Tree:
[0 0 0 0 0]
```

Penjelasan codingan:

1. tree_model = Pipeline(steps=[...])

Digunakan untuk membuat model Decision Tree yang menggabungkan preprocessing dan algoritma klasifikasi.

2. ("preprocessor", preprocessor)

Menunjukkan bahwa data harus melewati proses preprocessing sebelum digunakan dalam pemodelan.

3. DecisionTreeClassifier()

Digunakan untuk membangun algoritma klasifikasi berbasis pohon keputusan. Model ini bekerja dengan membagi data berdasarkan aturan tertentu hingga menghasilkan keputusan pada setiap cabang.

4. max_depth=8

Menentukan kedalaman maksimum pohon keputusan sebanyak 8 tingkat. Pembatasan ini digunakan untuk mengendalikan kompleksitas model.

5. min_samples_split=20

Menentukan jumlah minimum 20 sampel agar suatu simpul dapat dibagi menjadi cabang berikutnya.

6. min_samples_leaf=10

Menentukan bahwa setiap simpul akhir harus memiliki minimal 10 sampel.

7. class_weight="balanced"

Digunakan untuk memberikan bobot yang disesuaikan dengan jumlah data pada setiap kelas. Kelas yang jumlahnya lebih sedikit akan mendapatkan bobot lebih besar.

Pengaturan ini penting karena dataset memiliki ketidakseimbangan antara kelas no dan yes.

8. random_state=42

Digunakan agar hasil pelatihan dapat direproduksi secara konsisten.

9. tree_model.fit(X_train, y_train)

Digunakan untuk melatih model Decision Tree menggunakan data latih.

10. tree_model.predict(X_test)

Digunakan untuk menghasilkan prediksi kelas berdasarkan data uji.

11. tree_pred[:5]

Digunakan untuk menampilkan lima hasil prediksi pertama.

Pembahasan Output:

Model Decision Tree menghasilkan prediksi secara langsung dalam bentuk kelas 0 dan 1. Berbeda dengan Linear Regression yang menghasilkan nilai kontinu, Decision Tree menentukan kelas berdasarkan aturan keputusan yang dipelajari dari data latih.

Penggunaan class_weight="balanced" bertujuan membantu model mengenali kelas minoritas, yaitu nasabah yang menerima penawaran deposito.

Langkah 15. Evaluasi Model Decision Tree

In [20]:
```
# 202431076_AimeeJehovaJireh

tree_accuracy = accuracy_score(y_test, tree_pred)

tree_precision = precision_score(
    y_test, tree_pred, zero_division=0
)

tree_recall = recall_score(
    y_test, tree_pred, zero_division=0
)

tree_f1 = f1_score(
    y_test, tree_pred, zero_division=0
)

print("EVALUASI DECISION TREE")
print("--------------------------------")
print("Accuracy :", tree_accuracy)
print("Precision:", tree_precision)
print("Recall   :", tree_recall)
print("F1-Score :", tree_f1)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        tree_pred,
        target_names=["no", "yes"],
        zero_division=0
    )
)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, tree_pred))
```

```
EVALUASI DECISION TREE
--------------------------------
Accuracy : 0.8324671016255667
Precision: 0.35955746773202213
Recall   : 0.552930056710775
F1-Score : 0.43575418994413406

Classification Report:
              precision    recall  f1-score   support

          no       0.94      0.87      0.90      7985
         yes       0.36      0.55      0.44      1058

    accuracy                           0.83      9043
   macro avg       0.65      0.71      0.67      9043
weighted avg       0.87      0.83      0.85      9043


Confusion Matrix:
[[6943 1042]
 [ 473  585]]
```

Penjelasan codingan:

1. accuracy_score(y_test, tree_pred)

Digunakan untuk menghitung tingkat ketepatan seluruh hasil prediksi Decision Tree.

2. precision_score(y_test, tree_pred, zero_division=0)

Digunakan untuk mengetahui seberapa tepat model ketika memprediksi nasabah yang menerima penawaran deposito.

3. recall_score(y_test, tree_pred, zero_division=0)

Digunakan untuk mengetahui seberapa banyak nasabah yang benar-benar menerima penawaran berhasil ditemukan oleh model.

4. f1_score(y_test, tree_pred, zero_division=0)

Digunakan untuk menghitung keseimbangan antara precision dan recall.

5. classification_report()

Digunakan untuk menampilkan laporan evaluasi yang lebih lengkap berdasarkan masing-masing kelas.

6. y_test

Merupakan label aktual yang digunakan sebagai pembanding hasil prediksi.

7. tree_pred

Merupakan hasil prediksi yang dihasilkan oleh model Decision Tree.

8. target_names=["no", "yes"]

Digunakan untuk memberikan nama pada kelas 0 dan kelas 1 agar laporan lebih mudah dibaca.

9. confusion_matrix(y_test, tree_pred)

Digunakan untuk menghasilkan tabel yang menunjukkan jumlah prediksi benar dan salah pada setiap kelas.

Pembahasan Output:

Hasil evaluasi menunjukkan kemampuan model dalam melakukan klasifikasi terhadap keputusan nasabah. Classification Report memberikan informasi mengenai kinerja model pada kelas no maupun yes.

Sementara itu, Confusion Matrix digunakan untuk mengetahui jumlah data yang berhasil diklasifikasikan dengan benar dan jumlah data yang mengalami kesalahan klasifikasi.

Dengan demikian, evaluasi ini tidak hanya melihat ketepatan keseluruhan model, tetapi juga memperhatikan kemampuan model dalam mengenali nasabah yang menerima penawaran deposito.

Langkah 16. Visualisasi Confusion Matrix

```
In [22]: # 202431076_AimeeJehovaJireh

# Menghitung Confusion Matrix masing-masing model
cm_linear = confusion_matrix(y_test, linear_class)
cm_tree = confusion_matrix(y_test, tree_pred)

# Membuat area visualisasi
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Confusion Matrix Linear Regression
disp_linear = ConfusionMatrixDisplay(
    confusion_matrix=cm_linear,
    display_labels=["no", "yes"]
)

disp_linear.plot(
    ax=axes[0],
    colorbar=False,
    cmap="Blues"
)
```

```
axes[0].set_title("Linear Regression")

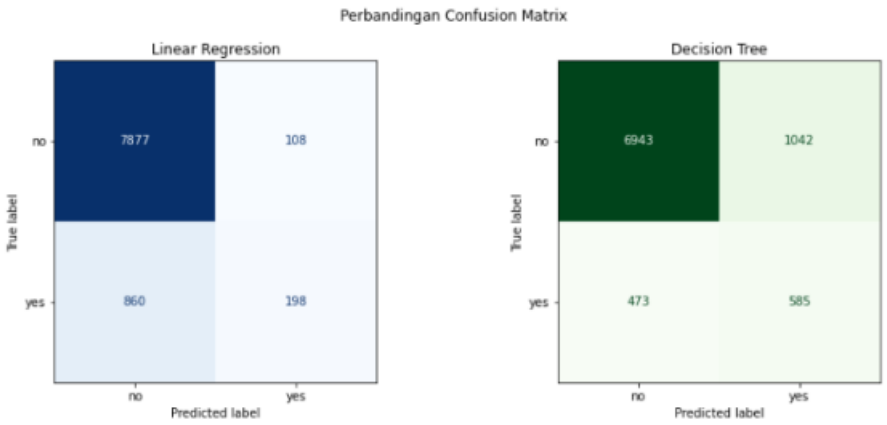
# Confusion Matrix Decision Tree
disp_tree = ConfusionMatrixDisplay(
    confusion_matrix=cm_tree,
    display_labels=["no", "yes"]
)

disp_tree.plot(
    ax=axes[1],
    colorbar=False,
    cmap="Greens"
)

axes[1].set_title("Decision Tree")

# Judul utama
plt.suptitle("Perbandingan Confusion Matrix")

plt.tight_layout()
plt.show()
```



Penjelasan setiap coding

- confusion_matrix(y_test, linear_class) digunakan untuk menghitung Confusion Matrix berdasarkan label aktual dan hasil prediksi Linear Regression.
- confusion_matrix(y_test, tree_pred) digunakan untuk menghitung Confusion Matrix berdasarkan label aktual dan hasil prediksi Decision Tree.
- plt.subplots(1, 2, figsize=(12, 5)) digunakan untuk membuat dua area grafik dalam satu baris dengan ukuran 12 × 5 inci.

- ConfusionMatrixDisplay() digunakan untuk membuat objek visualisasi Confusion Matrix berdasarkan hasil perhitungan sebelumnya.
- confusion_matrix=cm_linear memasukkan hasil Confusion Matrix Linear Regression ke dalam objek visualisasi.
- confusion_matrix=cm_tree memasukkan hasil Confusion Matrix Decision Tree ke dalam objek visualisasi.
- display_labels=["no", "yes"] digunakan untuk memberikan nama kategori pada grafik.
- disp_linear.plot() digunakan untuk menampilkan Confusion Matrix Linear Regression pada area grafik pertama.
- disp_tree.plot() digunakan untuk menampilkan Confusion Matrix Decision Tree pada area grafik kedua.
- ax=axes[0] dan ax=axes[1] menentukan posisi masing-masing grafik.
- colorbar=False digunakan untuk menghilangkan skala warna tambahan di samping grafik.
- cmap="Blues" dan cmap="Greens" digunakan untuk membedakan warna visualisasi kedua model.
- set_title() digunakan untuk memberikan judul pada setiap grafik.
- plt.suptitle() digunakan untuk memberikan judul utama visualisasi.
- plt.tight_layout() digunakan untuk merapikan tata letak grafik.
- plt.show() digunakan untuk menampilkan hasil visualisasi.

Pembahasan output untuk laporan

Visualisasi Confusion Matrix digunakan untuk memperlihatkan hasil klasifikasi dari masing-masing algoritma, yaitu Linear Regression dan Decision Tree. Kedua grafik menampilkan perbandingan antara hasil prediksi model dengan kategori aktual pada data uji.

Confusion Matrix terdiri dari empat komponen utama:

Komponen	Penjelasan
True Negative (TN)	Nasabah no diprediksi no dengan benar.
False Positive (FP)	Nasabah no diprediksi yes.
False Negative (FN)	Nasabah yes diprediksi no.
True Positive (TP)	Nasabah yes diprediksi yes dengan benar.

Melalui visualisasi tersebut, dapat diketahui jumlah prediksi yang benar maupun salah pada setiap kelas. Perbandingan kedua grafik juga membantu mengetahui kemampuan masing-masing algoritma dalam mengenali nasabah yang menerima dan menolak penawaran deposito berjangka.

Langkah 17. Perbandingan Hasil Linear Regression dan Decision Tree

In [26]:

```python
# Watermark NIM: 202431076_AimeeJehovaJireh

# Membuat tabel perbandingan hasil evaluasi
hasil_perbandingan = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Decision Tree"
    ],
    "Accuracy": [
        linear_accuracy,
        tree_accuracy
    ],
    "Precision": [
        linear_precision,
        tree_precision
    ],
    "Recall": [
        linear_recall,
        tree_recall
    ],
    "F1-Score": [
        linear_f1,
        tree_f1
    ]
})

# Menampilkan tabel perbandingan
print("TABEL PERBANDINGAN HASIL EVALUASI MODEL")
display(hasil_perbandingan.round(4))


# Membandingkan nilai setiap metrik
print("\nANALISIS HASIL PERBANDINGAN")
print("=" * 45)

metrics = ["Accuracy", "Precision", "Recall", "F1-Score"]

for metric in metrics:
    nilai_linear = hasil_perbandingan.loc[
        0, metric
    ]

    nilai_tree = hasil_perbandingan.loc[
        1, metric
    ]

    if nilai_linear > nilai_tree:
        terbaik = "Linear Regression"
    elif nilai_tree > nilai_linear:
        terbaik = "Decision Tree"
    else:
        terbaik = "Keduanya memiliki nilai yang sama"

    print(f"{metric}: {terbaik}")


# Menentukan model berdasarkan F1-Score
if linear_f1 > tree_f1:
    model_terbaik = "Linear Regression"
elif tree_f1 > linear_f1:
    model_terbaik = "Decision Tree"
else:
    model_terbaik = "Keduanya memiliki nilai F1-Score yang sama"

print("\nKESIMPULAN:")
print("Model dengan F1-Score tertinggi adalah:", model_terbaik)
```

```
TABEL PERBANDINGAN HASIL EVALUASI MODEL
```

	Model	Accuracy	Precision	Recall	F1-Score
0	Linear Regression	0.8930	0.6471	0.1871	0.2903
1	Decision Tree	0.8325	0.3596	0.5529	0.4358

```
ANALISIS HASIL PERBANDINGAN
=============================================
Accuracy: Linear Regression
Precision: Linear Regression
Recall: Decision Tree
F1-Score: Decision Tree

KESIMPULAN:
Model dengan F1-Score tertinggi adalah: Decision Tree
```

Penjelasan codingan:

- metrics = ["Accuracy", "Precision", "Recall", "F1-Score"] digunakan untuk menentukan metrik yang akan dibandingkan.
- for metric in metrics digunakan untuk melakukan perulangan terhadap setiap metrik evaluasi.
- hasil_perbandingan.loc[0, metric] mengambil nilai evaluasi Linear Regression dari baris pertama tabel.
- hasil_perbandingan.loc[1, metric] mengambil nilai evaluasi Decision Tree dari baris kedua tabel.
- if, elif, dan else digunakan untuk menentukan algoritma yang memperoleh nilai lebih tinggi pada setiap metrik.
- model_terbaik digunakan untuk menyimpan nama algoritma dengan nilai F1-Score tertinggi.
- print() digunakan untuk menampilkan hasil analisis dan kesimpulan perbandingan model.

Penjelasan Output:

Berdasarkan tabel perbandingan dan analisis hasil evaluasi, kedua algoritma menunjukkan kemampuan yang berbeda dalam melakukan klasifikasi keputusan nasabah. Linear Regression memperoleh nilai Accuracy dan Precision yang lebih tinggi, sedangkan Decision Tree memperoleh nilai Recall dan F1-Score yang lebih tinggi.

Nilai Recall yang lebih tinggi pada Decision Tree menunjukkan bahwa algoritma tersebut lebih mampu menemukan nasabah yang benar-benar menerima penawaran deposito. Sementara itu, nilai F1-Score yang lebih tinggi menunjukkan keseimbangan Precision dan Recall yang lebih baik.

Dengan demikian, berdasarkan hasil pengujian sebelumnya, Decision Tree menjadi model yang lebih sesuai apabila tujuan klasifikasi adalah mengenali nasabah yang berpotensi menerima penawaran deposito. Meskipun demikian, pemilihan model tetap perlu mempertimbangkan tujuan analisis dan karakteristik dataset.

BAB IV
PENUTUP

Berdasarkan landasan teori dan hasil praktikum pembelajaran mesin yang telah dilakukan menggunakan dataset Bank Customer, dapat diperoleh beberapa kesimpulan sebagai berikut:

1. Pembelajaran mesin (Machine Learning) merupakan salah satu metode dalam bidang kecerdasan buatan yang memungkinkan komputer mempelajari pola dari data untuk menghasilkan prediksi atau keputusan. Dalam praktikum ini, pembelajaran mesin diterapkan untuk memprediksi keputusan nasabah dalam menerima atau menolak penawaran deposito berjangka berdasarkan karakteristik dan riwayat aktivitas nasabah.

2. Dataset Bank Customer yang digunakan terdiri dari 45.211 baris data dan 17 kolom. Berdasarkan hasil analisis awal, diketahui bahwa dataset memiliki dua kategori target, yaitu no sebanyak 39.922 data dan yes sebanyak 5.289 data. Perbedaan jumlah tersebut menunjukkan adanya ketidakseimbangan kelas (class imbalance) yang perlu diperhatikan dalam proses pembangunan dan evaluasi model.

3. Proses Exploratory Data Analysis (EDA) membantu memahami karakteristik dataset melalui analisis statistik deskriptif dan visualisasi. Fitur numerik seperti age, balance, day, duration, campaign, pdays, dan previous dianalisis untuk mengetahui pola persebaran datanya. Sementara itu, fitur kategorikal dianalisis untuk mengetahui variasi kategori yang terdapat pada karakteristik nasabah.

4. Tahapan preprocessing dilakukan untuk mempersiapkan data sebelum digunakan dalam pemodelan. Proses ini meliputi pemisahan fitur dan target, pembagian data menjadi data latih dan data uji, penanganan nilai kosong, standardisasi fitur numerik, serta perubahan fitur kategorikal menggunakan One-Hot Encoding. Kolom duration tidak digunakan dalam pemodelan karena nilainya baru diketahui setelah percakapan dengan nasabah selesai, sehingga berpotensi menyebabkan data leakage apabila prediksi dilakukan sebelum panggilan berlangsung.

5. Algoritma Linear Regression dan Decision Tree berhasil diterapkan untuk memprediksi keputusan nasabah. Linear Regression digunakan sebagai pendekatan regresi dengan target biner yang kemudian dikonversi menjadi kelas menggunakan ambang batas 0,5. Sementara itu, Decision Tree melakukan klasifikasi berdasarkan aturan keputusan yang terbentuk dari pola data latih.

6. Berdasarkan hasil evaluasi, Linear Regression memperoleh nilai akurasi sekitar 89,30%, sedangkan Decision Tree memperoleh nilai akurasi sekitar 83,25%. Namun, Decision Tree memiliki kemampuan yang lebih baik dalam mengenali kelas yes, dengan nilai recall sekitar 55,29%, dibandingkan Linear Regression yang memiliki nilai recall sekitar 18,71%. Hal ini menunjukkan bahwa akurasi yang tinggi belum tentu menggambarkan kemampuan model dalam mengenali seluruh kelas, terutama pada dataset yang tidak seimbang.

7. Melalui praktikum ini, dapat dipahami bahwa keberhasilan penerapan pembelajaran mesin tidak hanya bergantung pada pemilihan algoritma, tetapi juga pada kualitas data, proses preprocessing, pemahaman karakteristik dataset, serta pemilihan metrik evaluasi yang sesuai. Oleh karena itu, perbandingan beberapa algoritma diperlukan agar dapat diketahui model yang lebih sesuai dengan tujuan prediksi.

Secara keseluruhan, praktikum ini memberikan pemahaman mengenai penerapan tahapan pembelajaran mesin mulai dari pengolahan data, analisis eksploratif, preprocessing, pembangunan model, hingga evaluasi hasil prediksi. Hasil yang diperoleh menunjukkan bahwa Linear Regression dan Decision Tree memiliki karakteristik kinerja yang berbeda dalam memprediksi keputusan nasabah, sehingga pemilihan model perlu mempertimbangkan kebutuhan dan tujuan klasifikasi yang ingin dicapai.

DAFTAR PUSTAKA

Delle Site, P., & Parmar, J. (2024). On the linear probability model as binary choice random utility model. Journal of Choice Modelling, 52, 100505. https://doi.org/10.1016/j.jocm.2024.100505.

Lee, M., Lee, G., & Choi, J. (2023). Linear probability model revisited: Why it works and how it should be specified. Sociological Methods & Research, 54(1), 173–186. https://doi.org/10.1177/00491241231176850.

Maulidah, N. (2023). Prediksi peningkatan jumlah nasabah deposito berjangka menggunakan algoritma KNN, Decision Tree, Random Forest dan XGBoost. InComTech: Jurnal Telekomunikasi dan Komputer, 13(2), 90–95. https://doi.org/10.22441/incomtech.v13i2.16921.

Nasir, F., Ahmed, A. A., Kiraz, M. S., Yevseyeva, I., & Saif, M. (2024). Data-driven decision-making for bank target marketing using supervised learning classifiers on imbalanced big data. Computers, Materials & Continua, 81(1), 1703–1728. https://doi.org/10.32604/cmc.2024.055192.

Olajide, B. O., & Andrew, I. W. (2023). Bank term deposit service patronage forecasting using machine learning. Vallis Aurea, 9(2), 53–64. https://doi.org/10.2507/IJVA.9.2.5.106.

Watono, B., Utami, E., & Ariatmanto, D. (2024). Stacking ensemble learning and instance hardness threshold for bank term deposit acceptance classification on imbalanced dataset. Jurnal Teknik Informatika (JUTIF), 5(4), 453–467. https://doi.org/10.52436/1.jutif.2024.5.4.2022.